CS 313 - Project 2
Max-Heap Priority Queue

Deadline: 11:59PM, Wednesday, November 5, 2025

1 Project Overview

For this lab, we will be taking what we've learned so far up a notch and learn how to implement a max-heap priority queue. Priority queues are another commonly used Abstract Data Type (ADT) due to the fact that they are self-sorting (they sort themselves) and that their time complexity ranges from constant $O(1)$ to linear $O(n)$ depending on the operation and how they are implemented. Our central task for this lab is to implement the max-heap variety of priority queues.

2 Program Requirements:

2.1 Write the priority queue class.

You will need to write a **PriorityQueue** class. To receive any credit, your class must follow the specifications below exactly:

- Class Name: **PriorityQueue**
- Methods:
  - $\_\_init\_\_(< PriorityQueue > self, < Int > capacity) \rightarrow < Nonetype > None$
    - **[5 points]**
    - **Complexity:** $O(1)$.
    - **Valid Input:** An integer from $(0, \infty]$.
    - Instance variables:
      - $< list > \_heap$ This is where the heap will be stored. Our heap takes the form of a list.
      - $< Int > capacity$: This variable contains the total number of items that can be fit into the queue.
      - $< Int > currentSize$: This variable contains the current number of items in the queue.
      - Any other instance variables you want.
  - $isEmpty(< PriorityQueue > self) \rightarrow < Bool > returnValue$
    - **[5 points]**
    - **Complexity:** $O(1)$.
    - **Description:** This method returns True/False depending on if the priority queue is empty or not.
  - $isFull(< PriorityQueue > self) \rightarrow < Bool > returnValue$
    - **[5 points]**
    - **Complexity:** $O(1)$.
    - **Description:** This method returns True/False depending on if the priority queue is full or not.

- $getParent(< PriorityQueue > self, < Int > pos) \rightarrow < Int > returnValue$
  - **[5 points]**
  - **Complexity:** $O(1)$.
  - **Valid Input:** An integer indicating the position of the current item.
  - **Description:** This method computes and returns the parent position for the current item.
- $getLeftChild(< PriorityQueue > self, < Int > pos) \rightarrow < Int > returnValue$
  - **[5 points]**
  - **Complexity:** $O(1)$.
  - **Valid Input:** An integer indicating the position of the current item.
  - **Description:** This method computes the position of the left child for the current item.
- $getRightChild(< PriorityQueue > self, < Int > pos) \rightarrow < Int > returnValue$
  - **[5 points]**
  - **Complexity:** $O(1)$.
  - **Valid Input:** An integer indicating the position of the current item.
  - **Description:** This method computes the position of the right child for the current item.
- $swap(< PriorityQueue > self, < Int > childPos, < Int > parentPos) \rightarrow < Nonetype > None$
  - **[10 points]**
  - **Complexity:** $O(1)$.
  - **Valid Input:** Two integers indicating the positions of the child and the parent to be swapped.
  - **Description:** This method swaps the positions of the provided child and parent to each other in place without returning anything.
- $heapify(< PriorityQueue > self, < Int > index) \rightarrow < Bool > returnValue$
  - **[15 points]**
  - **Complexity:** $O(log(n))$.
  - **Valid Input:** An integer indicating the starting position to reshape the heap.
  - **Description:** This method reshapes the structure of the heap after an extraction of a max node.
- $insert(< PriorityQueue > self, < Tuple > item) \rightarrow < Bool > returnValue$
  - **[20 points]**
  - **Complexity:** $O(log(n))$.
  - **Valid Input:** A tuple $(priority, item)$ containing the following:
    - $priority$: A positive integer in the bound $(0, \infty]$.
    - $item$: Any Python object.
  - **Description:** This method adds a tuple to the queue based on its priority, then returns True upon successfully adding it to the heap. If any other error happens, raise either **QueueIsFull** or **InvalidInputTuple** as required above.
  - **Note:** When adding a node to your heap, remember that for every position i, the priority of i must be greater than or equal to the priorities of its children, but your heap must also maintain the correct shape. (i.e., for any position i there can be at most two children, and the parent has a greater priority than all subsequent nodes.
- $extractMax(< PriorityQueue > self) \rightarrow < Tuple > returnValue$
  - **[20 points]**
  - **Complexity:** $O(log(n))$.
  - **Description:** This method removes and returns the tuple with the highest priority. **Note:** Do not forget to reorder the heap after extraction. (i.e., call $maxHeapify$)
- $peekMax(< PriorityQueue > self) \rightarrow < Tuple > returnValue$

* [**5 points**]
* **Complexity:** $O(1)$.
* **Description:** This method returns the tuple with the highest priority if the queue is not empty. Otherwise, it will return False and not raise exceptions.

3 Submission Requirements:

Submit a single file **lab2.py** to canvas.